

Compiled by **Senior Dev: Austine Alex**

CoBrain Forecasting Engine Design Specs

The complete technical specification for the multi-tenant Temporal Fusion forecasting system — architecture, mathematics, training data, and executable model design across the Universal, Industry, and Geography tiers.

THIS DOCUMENT CONTAINS

- The corrected system architecture (executable)
- All mathematical formulae, explained
- Model component specifications
- Feature engineering — events to tensors
- Normalisation rules for multi-tenant training
- Executable code (PyTorch)
- Where to find training data
- Folder structure and build order

▪ Contents

1 · System Overview & Design Philosophy	03
2 · The Corrected Architecture (seven fixes)	04
3 · Component 1 — Universal Sequence Encoder	06
4 · Component 2 — Industry Mask & Projector	07
5 · Component 3 — Geography (static + time-varying)	07
6 · Component 4 — Dynamic Trajectory Fusion Engine	08
7 · Component 5 — Quantile Predictor	08
8 · Component 6 — Static Covariate Encoder	09
9 · The Mathematics, Fully Explained	10
10 · Feature Engineering — Events to Tensors	12
11 · Normalisation — The Six Rules	15
12 · Executable Model (PyTorch)	17
13 · Executable Feature Pipeline	19
14 · Where to Find Training Data	21
15 · Tenant Isolation & Cold-Start	23
16 · Folder Structure & Build Order	24

1 · System Overview & Design Philosophy

CoBrain's forecasting engine predicts organisational trajectories — burnout risk, delivery velocity, cash-flow, churn, and more — across many companies spanning different industries and countries, using a single conditioned model.

The four load-bearing decisions

Decision 1 — One conditioned model, not one model per tenant

A single sequence model serves every company. Company identity enters through conditioning signals (industry, geography) rather than separate models. This gives **cross-tenant learning** — patterns learned from the whole fleet improve every tenant's forecast — while keeping per-tenant specialisation.

Decision 2 — Three tiers, three distinct mechanisms

Each tier operates differently because each tier *is* different in nature:

TIER	MECHANISM	PRODUCES
1 · Universal	Sequence encoder	Base feature trajectory z_t
2 · Industry	Feature mask + projector	Feature mask m_{ind}
3 · Geography	Latent projector (static) + known-future channel	Geo latent e_{geo} + time-varying geo signals

Decision 3 — Quantile output, always

The model never emits a single number. It emits three quantiles — Bear (P10), Base (P50), Bull (P90) — so the forecast band communicates honest uncertainty. In volatile markets, band width is itself the insight.

Decision 4 — Tenant isolation by namespace

Every store is namespaced per tenant. No tenant's raw data or trajectory is ever visible to another. Isolation is enforced at the data layer, not just the API.

THE CORE INSIGHT

Your three tiers map cleanly onto the three input channels a temporal fusion model consumes natively: **observed** (Tier 1 sequences), **static** (Industry mask + Geography identity), and **known-future** (scheduled geography signals). The architecture is the composition of these three.

2 · The Corrected Architecture

This is your uploaded design, corrected into an executable form. Seven fixes were applied. Five correct the architecture itself; two more (added in version 2) correct the output head and the conditioning path that makes the three-tier design actually function.

THE SEVEN FIXES APPLIED

1. `z_t` is now explicitly a per-timestep sequence tensor $[T \times d]$, broadcast at fusion. **2.** Industry mask `m_ind` gates the encoder INPUT (not its output). **3.** Geography split into static latent `e_geo` + time-varying known-future `g_t`. **4.** All stores labelled "per-tenant namespace". **5.** Fusion engine emits a second head — interpretability weights. **6.** The output head uses the last timestep, never Flatten. **7.** The static covariate encoder initialises the LSTM hidden state.

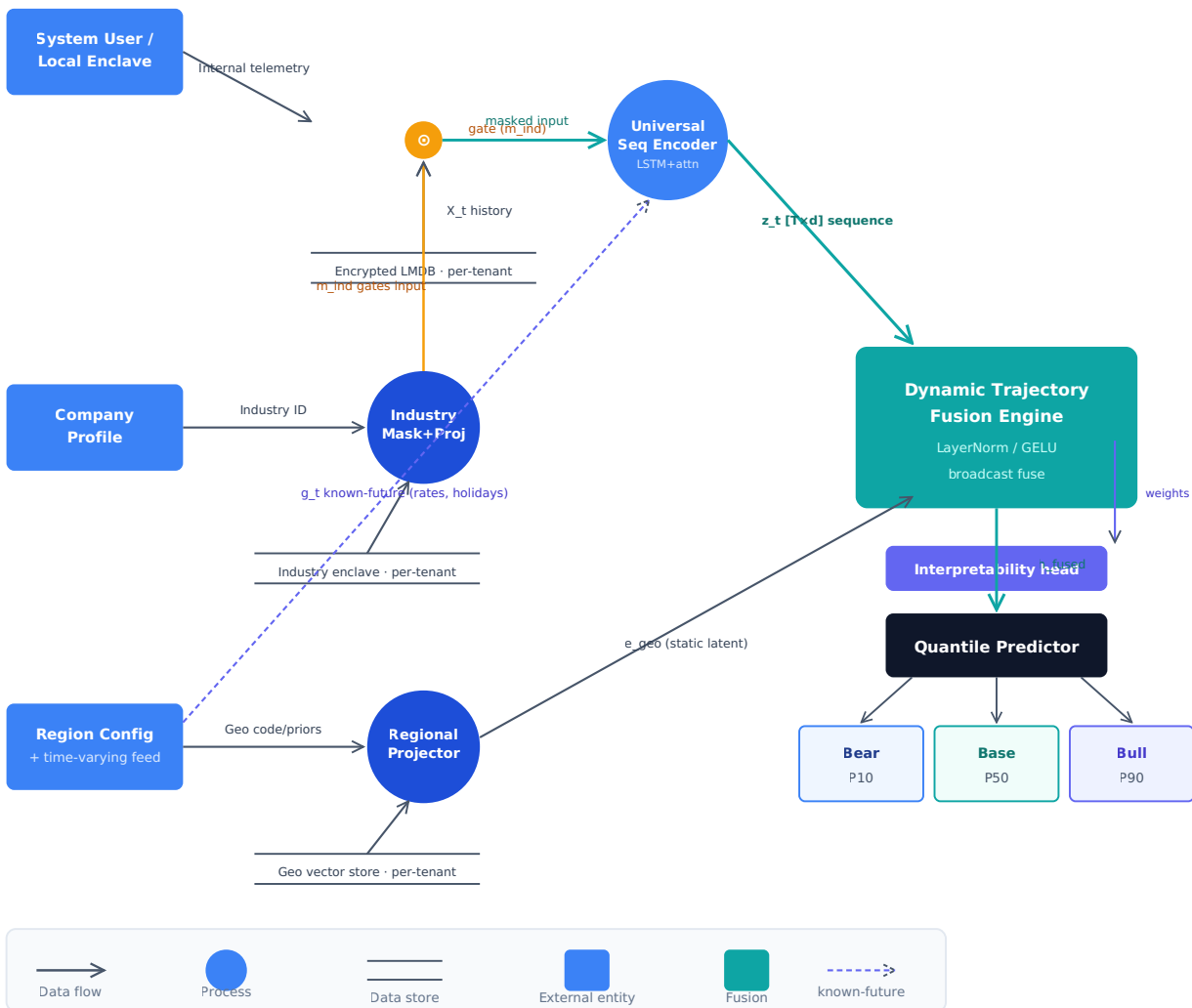
FIX 6 — WHY FLATTEN IS THE WRONG HEAD

A common time-series transformer tutorial flattens the $[T \times d]$ sequence into one long vector before the output layer, because it predicts a single scalar. That is wrong here for three reasons: it explodes the parameter count, it hard-locks `seq_len` so the lookback window can never vary, and it destroys the temporal structure the encoder just built. Use the last timestep's hidden state, `h[:, -1, :]`, and project it to the full horizon.

FIX 7 — THE CONDITIONING PATH THAT MAKES THE TIERS REAL

The static covariate encoder must produce four context vectors — `c_h` and `c_c` initialise the LSTM hidden and cell state, `c_sel` conditions variable selection, and `c_enrich` conditions attention enrichment. Without this wiring, industry and geography are just extra input columns and the entire three-tier architecture stops doing anything. This is the single most important structural correction.

CoBrain Forecasting Engine — Corrected Executable Architecture



Reading the corrected flow

Internal telemetry and the sequence history X_t meet the industry mask m_{ind} at a gating junction **before** the encoder — so industry-irrelevant features are suppressed at the input, not after contamination. The Universal Encoder emits z_t as an explicit per-timestep sequence. Geography contributes twice: a static latent e_{geo} into the fusion engine, and a time-varying known-future stream g_t (scheduled rate decisions, holidays) into the encoder. The Fusion Engine broadcasts the static signals across the sequence, then emits both the fused representation and a set of interpretability weights. The Quantile Predictor produces Bear/Base/Bull.

3 · Component 1 — Universal Sequence Encoder

Tier 1. Consumes the time-series history of internal signals and produces the base feature trajectory z_t — a per-timestep encoded representation of the company's recent state.

Inputs

- X_t — the observed sequence: comms volume, after-hours ratio, sentiment, task throughput, PR/issue velocity, doc-creation rate. Shape $[T, F]$ (T timesteps, F features).
- g_t — time-varying known-future geography signals (interest-rate path, holiday calendar). Concatenated to X_t .
- m_{ind} — the industry mask, applied element-wise to the input features BEFORE encoding.

Mechanism

A recurrent encoder (LSTM) processes the masked input sequence, followed by interpretable self-attention across timesteps. The output is z_t of shape $[T, d]$ where d is the hidden dimension.

$$z_t = \text{Attention}(\text{LSTM}(m_{ind} \odot [X_t ; g_t]))$$

$\odot$ = element-wise gate · $[:,]$ = concatenation · output shape $[T, d]$

WHY MASK AT INPUT (FIX 2)

If the industry mask were applied after encoding, industry-irrelevant features would already be baked into z_t . Masking the input means a finance company's encoder never even sees tech-specific signals — they are zeroed before the LSTM. The representation is clean per industry.

4 · Component 2 — Industry Mask & Projector

Tier 2. Takes the industry type ID and produces a feature mask that gates which input signals are relevant for that industry.

Mechanism

The industry ID indexes an embedding, which a small projector maps to a mask vector the same width as the input features. A sigmoid keeps mask values in $[0,1]$ — soft gating, so the model can learn partial relevance rather than hard on/off.

$$m_{ind} = \sigma(W_m \cdot \text{Embed}(\text{industry}) + b_m)$$

σ = sigmoid · values in $[0,1]$ · one weight per input feature

INDUSTRY	HIGH MASK WEIGHT ON	LOW MASK WEIGHT ON
Finance	transaction volume, cash signals	PR velocity, deploy frequency
Tech	deploy frequency, issue velocity	transaction/AUM signals
Retail	inventory, seasonal demand	code-related signals

5 · Component 3 — Geography (Static + Time-Varying)

Tier 3. The fix here is critical — geography is NOT purely static. It contributes both a static identity latent and a time-varying known-future stream.

Static half — e_{geo}

The geo code (country) indexes an embedding projected to a latent vector that captures durable country identity — "this is Kenya", carrying learned priors about that market.

$$e_{geo} = W_g \cdot \text{Embed}(\text{geo_code}) + b_g$$

static per tenant · fused into every timestep by broadcast

Time-varying half — g_t (Fix 3)

Scheduled and observed country signals that change over time: interest-rate decision dates, inflation trajectory, FX volatility, holidays, elections. These are known ahead for future timesteps and feed directly into the encoder as known-future inputs.

WHY GEOGRAPHY NEEDS BOTH HALVES

A static embedding captures "this company is in Kenya" but not "Kenya's central bank meets next Tuesday". A hiring forecast in a 12%-inflation, high-FX-volatility market must react to the *trajectory* of those signals, not just the country label. The static latent gives identity; the time-varying stream gives dynamics. Both are required.

6 · Component 4 — Dynamic Trajectory Fusion Engine

The convergence point. Combines three heterogeneous representations — the sequence trajectory z_t , the static geo latent e_{geo} , and the industry context — into a single fused representation, and emits interpretability weights.

Mechanism

The static signals (e_{geo} , industry context) are broadcast across every timestep of z_t , concatenated, then passed through a gated feed-forward block with LayerNorm and GELU. This is where the "broadcast fuse" happens — each timestep of the trajectory is contextualised by the company's static identity.

$$h_t = \text{LayerNorm}(z_t + \text{GELU}(W_f \cdot [z_t ; e_{geo} ; c_{ind}]))$$

e_{geo}, c_{ind} broadcast across all T timesteps · h_t shape [T, d]

The interpretability head (Fix 5)

A parallel projection produces attention/selection weights indicating which inputs drove the forecast — the "why" signal that becomes a competitive advantage in the product UI.

$$\alpha = \text{softmax}(W_\alpha \cdot h_t)$$

α = per-feature importance weights, surfaced to the user

7 · Component 5 — Quantile Predictor

Takes the fused representation and projects it to three quantiles per future timestep, producing the Bear / Base / Bull trajectory.

$$\hat{y}_{\tau(q)} = W_q \cdot h_T + b_q \quad \text{for } q \in \{0.1, 0.5, 0.9\}$$

three linear heads, one per quantile, over the forecast horizon

QUANTILE	CASE	INTERPRETATION
P10	Bear	Pessimistic — only 10% chance the true value is below this
P50	Base	Median — the central forecast
P90	Bull	Optimistic — only 10% chance the true value is above this

9 · The Mathematics, Fully Explained

9.1 Quantile Loss — the training objective

The model is trained to minimise quantile loss (pinball loss). For a true value y , prediction $\hat{y}$ at quantile q :

$$QL(y, \hat{y}, q) = q \cdot \max(y - \hat{y}, 0) + (1 - q) \cdot \max(\hat{y} - y, 0)$$

asymmetric penalty — controls which side of the error is punished harder

Intuition: for the P90 head ($q=0.9$), under-predicting ($y > \hat{y}$) is penalised 9× more than over-predicting. This pushes the P90 line upward until it genuinely covers the high-scenario tail. The P10 head does the opposite. This is what produces honest, asymmetric bands. The total loss sums over all quantiles and timesteps:

$$L = \sum_q \sum_t QL(y_t, \hat{y}_t(q), q)$$

9.2 The Gated Linear Unit (GLU)

The gating mechanism throughout the fusion engine. It lets the network suppress irrelevant components entirely by multiplying them toward zero — critical when a tenant has sparse data.

$$GLU(a) = \sigma(W_1 \cdot a + b_1) \odot (W_2 \cdot a + b_2)$$

σ = sigmoid gate · $\odot$ = element-wise product

9.3 The Gated Residual Network (GRN)

The core reusable block. Wraps a nonlinear transform in a gate and a residual connection, so information can flow through unchanged when the transform isn't needed.

$$GRN(a) = LayerNorm(a + GLU(W_3 \cdot ELU(W_4 \cdot a + b_4) + b_3))$$

residual + gate = the network can learn to skip this block entirely

9.4 Interpretable Multi-Head Attention

How the encoder decides which past timesteps matter for the forecast. Standard scaled dot-product attention, modified so value weights are shared across heads — which makes the resulting attention weights directly interpretable as temporal importance.

$$Attention(Q, K, V) = softmax(Q K^T / \sqrt{d_k}) \cdot V$$

the softmax weights reveal which historical weeks drove the prediction

9.5 L2 normalisation of the geo latent

Before fusion, the geo latent is L2-normalised so no single country embedding dominates by magnitude — only its direction (learned identity) carries information.

$$\hat{e}_{geo} = e_{geo} / ||e_{geo}||_2$$

unit vector · stabilises fusion across countries of different data volumes

8 · Component 6 — Static Covariate Encoder

The conditioning engine. Takes the tenant's identity — industry, country, size band, currency — and produces four context vectors that condition every other component. This is the mechanism by which the three tiers actually influence the forecast.

Inputs

A dictionary of categorical IDs, one per static covariate. Each is embedded, the embeddings are concatenated, and four separate Gated Residual Networks project the result into four purpose-specific context vectors.

$$s = [\text{Embed}(\text{industry}) ; \text{Embed}(\text{country}) ; \text{Embed}(\text{size}) ; \text{Embed}(\text{currency})]$$

concatenated static embedding, shape [B, d_in]

$$c_h = \text{GRN}_1(s) \quad c_c = \text{GRN}_2(s) \quad c_sel = \text{GRN}_3(s) \quad c_enrich = \text{GRN}_4(s)$$

four independent projections, each shape [B, d_hidden]

What each context vector does

VECTOR	CONSUMED BY	EFFECT
c_h	LSTM initial hidden state	The sequence encoder starts from a tenant-specific state
c_c	LSTM initial cell state	Long-term memory is primed by tenant identity
c_sel	Variable Selection Network	Which features matter varies by industry and country
c_enrich	Attention enrichment GRN	Temporal attention is conditioned on identity

WHY THIS COMPONENT IS LOAD-BEARING

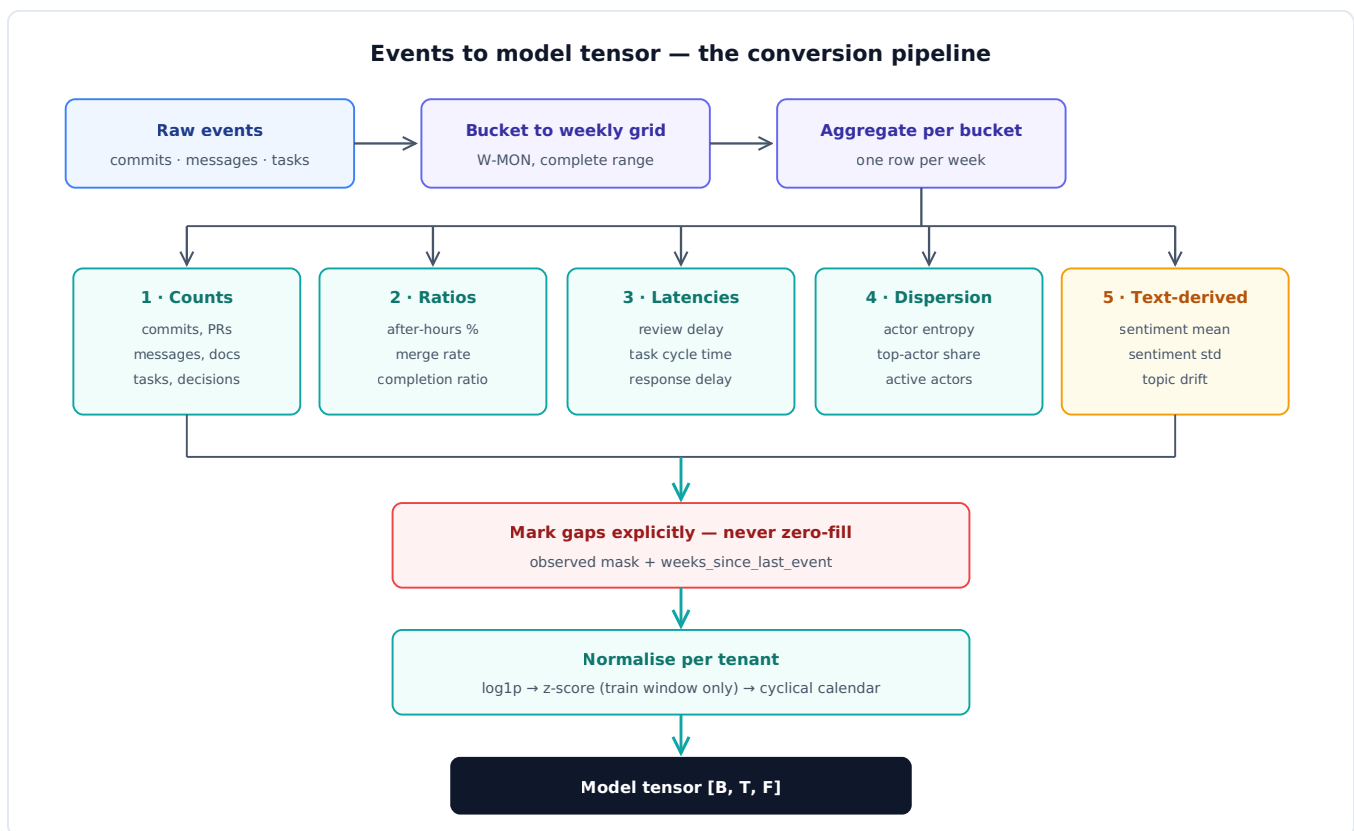
Remove this encoder and feed industry and country as ordinary input columns instead, and the model treats them as two more numbers among fifty. The tier structure collapses into a flat feature soup. With the encoder wired in, tenant identity reshapes the encoder's starting state, its feature selection, and its attention — which is precisely what "conditioned by covariates rather than separate models per tenant" means in practice.

10 · Feature Engineering — Events to Tensors

The hardest practical problem in this system: converting GitHub commits, Slack threads, Notion documents, decisions, and task allocations into the numeric tensors the model consumes. This section defines exactly how.

THE CORE PRINCIPLE

You never convert an individual event into a number. An event has a timestamp; a time series needs a value at every regular interval. The operation is therefore: **aggregate events onto a fixed weekly grid and compute features per bucket**. A commit is not a data point — a week's worth of commit behaviour is.



10.1 The five feature families

Family 1 — Counts (raw volume)

`n_commits`, `n_prs_opened`, `n_prs_merged`, `n_messages`, `n_docs_created`, `n_docs_edited`, `n_tasks_assigned`, `n_tasks_completed`, `n_decisions`. One integer per signal per bucket.

Family 2 — Ratios (scale-free, often more predictive than counts)

`after_hours_ratio`, `weekend_ratio`, `merge_rate` (`merged ÷ opened`), `completion_ratio` (`completed ÷ assigned`), `reopen_rate`. These are already normalised by construction, which makes them transfer well across tenants of different sizes.

Family 3 — Latencies (where timestamps become features)

median_review_delay (PR opened → first review), median_cycle_time (task assigned → completed), median_response_delay (message → first reply), oldest_open_task_age, weeks_since_last_decision. Rising cycle time is one of the strongest leading indicators of the delivery and burnout outcomes this system forecasts.

Family 4 — Dispersion (how concentrated the work is)

Compute Shannon entropy over each person's share of the week's events:

$$H = - \sum_i p_i \log p_i$$

p_i = person i 's share of that week's events

Low entropy means one person is carrying everything — a direct bus-factor and burnout signal. Also: top_actor_share and n_active_actors.

Family 5 — Text-derived (Slack and Notion content reduced to scalars)

sentiment_mean and sentiment_std from a small classifier over the week's messages, plus topic_drift — the cosine distance between this week's mean embedding and a rolling eight-week baseline centroid. Because the embeddings already exist in Qdrant for retrieval, topic drift is nearly free to compute and directly powers the topic-shift forecast.

DO NOT FEED RAW EMBEDDINGS AS VARIABLES

Passing a 384-dimension embedding into the model as 384 separate time-varying variables is a mistake. Variable selection treats each variable independently, so the network becomes expensive to train and impossible to interpret. Reduce text to a handful of meaningful scalars instead.

10.2 What must NOT be a static covariate

author_id, repo_id, and channel_id are not static covariates. Static means constant across the entire series for that entity — but the set of authors changes every week as people join, leave, and rotate. These fields are also high-cardinality and do not transfer across tenants: repo 7 in tenant A has nothing in common with repo 7 in tenant B, so a shared embedding over them is meaningless. Replace them entirely with the aggregate Family 4 dispersion features.

10.3 Bucket size

Weekly, matching the 13-week horizon. Hourly or 15-minute bucketing is a common instinct and is wrong here: it produces thousands of near-empty, extremely noisy timesteps, and the outcomes being forecast — burnout, velocity, knowledge gaps — simply do not resolve at that granularity. Daily is the finest defensible bucket, and only if the tenant volume is high enough to justify it.

11 · Normalisation — The Six Rules

This is where multi-tenant forecasting implementations most often fail silently. Each rule below prevents a specific, real failure mode. Rules 1 and 6 are the two that would quietly destroy the model without producing any visible error.

Rule 1 — Normalise per tenant, not globally

A ten-person startup produces around 50 commits a week; a 500-person company produces 5,000. Feed raw counts into a global model and it spends its capacity learning company *size* instead of organisational *dynamics* — and the cross-tenant learning that constitutes the entire competitive moat stops working. Z-score every feature against that tenant's own history, or convert to per-active-contributor rates.

Rule 2 — Log-transform counts before scaling

Event counts are heavy-tailed: a single release week can be ten times a normal week. Apply `log1p(x)` first, then z-score. Without this, a handful of outlier weeks dominate the scaling and compress all normal variation toward zero.

Rule 3 — Encode calendar time cyclically

Week 52 and week 1 are adjacent in reality but maximally distant as integers. Encode as a sine and cosine pair so the model sees the wrap-around correctly.

$$week_sin = \sin(2\pi w / 52) \quad week_cos = \cos(2\pi w / 52)$$

always used as a pair — one alone is ambiguous

Rule 4 — Categoricals become embeddings, never raw integers

Feeding `industry_id = 7` into a numeric channel implies industry 7 is "greater than" industry 3, which is meaningless. Every categorical — industry, country, size band, currency — goes through `nn.Embedding`.

Rule 5 — Missing buckets are explicit, never silently zero-filled

Zero means "no activity happened". Missing means "we did not observe". These are different facts and the model must be able to tell them apart. Emit two additional channels:

CHANNEL	MEANING
observed	Binary mask — 1 if the week had any data at all
weeks_since_last_event	Run-length of consecutive unobserved weeks

Carry-forward of continuous states has the same defect: it manufactures data that was never measured, and the model learns staleness as though it were signal.

Rule 6 — Fit normalisation statistics on the training window only

If the mean and standard deviation used for z-scoring are computed over a range that includes future weeks, the future has leaked backwards into the past. The backtest will look excellent and the production forecasts will be worthless. Fit on the training window, then apply those frozen statistics forward to validation and inference.

THE SILENT FAILURE

Rules 1 and 6 both fail without raising an error. The model trains, the loss decreases, the validation metrics look strong — and the deployed forecasts are meaningless. Assert both in tests: verify that normalisation statistics are tenant-scoped, and that no statistic was computed from a timestamp later than the training cut-off.

12 · Executable Model (PyTorch)

The architecture above, converted into runnable PyTorch. This is the skeleton that makes the diagram executable — each class maps to one component in the diagram.

12.1 Industry Mask & Projector

```
import torch, torch.nn as nn

class IndustryMask(nn.Module):
    # Tier 2 – produces a soft feature mask from industry ID
    def __init__(self, n_industries, n_features, emb=16):
        super().__init__()
        self.embed = nn.Embedding(n_industries, emb)
        self.proj = nn.Linear(emb, n_features)

    def forward(self, industry_id):
        e = self.embed(industry_id)
        return torch.sigmoid(self.proj(e)) # [B, n_features] in [0,1]
```

12.2 Universal Sequence Encoder (mask applied at input — Fix 2)

```
class UniversalEncoder(nn.Module):
    def __init__(self, n_features, hidden=64):
        super().__init__()
        self.lstm = nn.LSTM(n_features, hidden, batch_first=True)
        self.attn = nn.MultiheadAttention(hidden, num_heads=4, batch_first=True)

    def forward(self, x_seq, g_seq, m_ind):
        # x_seq [B,T,F] observed · g_seq [B,T,G] known-future · m_ind [B,F]
        x = torch.cat([x_seq, g_seq], dim=-1)
        m = torch.cat([m_ind, torch.ones_like(g_seq[:, 0, :])], dim=-1)
        x = x * m.unsqueeze(1) # GATE AT INPUT
        z, _ = self.lstm(x) # [B,T,d]
        z, attn_w = self.attn(z, z, z) # interpretable attn
        return z, attn_w # z_t is a SEQUENCE (Fix 1)
```

12.3 Geography Projector (static latent — Fix 3 time-varying feeds encoder)

```
class GeoProjector(nn.Module):
    def __init__(self, n_regions, latent=32, emb=16):
        super().__init__()
        self.embed = nn.Embedding(n_regions, emb)
        self.proj = nn.Linear(emb, latent)

    def forward(self, geo_code):
        e = self.proj(self.embed(geo_code))
        return nn.functional.normalize(e, dim=-1) # L2 norm
```

12.4 Fusion Engine + Quantile Predictor

```
class FusionEngine(nn.Module):
    def __init__(self, d, geo_dim, ind_dim):
        super().__init__()
        self.ff = nn.Linear(d + geo_dim + ind_dim, d)
        self.norm = nn.LayerNorm(d)
        self.act = nn.GELU()
        self.interp = nn.Linear(d, d)          # interpretability head (Fix 5)

    def forward(self, z, e_geo, c_ind):
        T = z.size(1)
        eg = e_geo.unsqueeze(1).expand(-1, T, -1)  # broadcast
        ci = c_ind.unsqueeze(1).expand(-1, T, -1)
        h = self.act(self.ff(torch.cat([z, eg, ci], dim=-1)))
        h = self.norm(z + h)
        alpha = torch.softmax(self.interp(h), dim=-1)
        return h, alpha

class QuantilePredictor(nn.Module):
    def __init__(self, d, horizon, quantiles=(0.1, 0.5, 0.9)):
        super().__init__()
        self.heads = nn.ModuleList([nn.Linear(d, horizon) for _ in quantiles])
        self.quantiles = quantiles

    def forward(self, h):
        last = h[:, -1, :]                          # [B,d]
        return {q: head(last) for q, head in zip(self.quantiles, self.heads)}
```

12.5 Quantile Loss

```
def quantile_loss(preds, target):
    # preds: dict{q: [B,H]}, target: [B,H]
    total = 0.0
    for q, yhat in preds.items():
        err = target - yhat
        total = total + torch.max(q * err, (q - 1) * err).mean()
    return total
```

THIS IS THE EXECUTABLE FORM OF YOUR DIAGRAM

Each class is one box in the corrected architecture. Wire them in order: IndustryMask → UniversalEncoder (masked input) → GeoProjector → FusionEngine → QuantilePredictor, trained with quantile_loss. Swap the LSTM for the full pytorch-forecasting TFT when you scale — the interfaces match.

13 · Executable Feature Pipeline

The data layer implementing Sections 10 and 11. These functions turn raw connector events into the normalised tensor the model consumes.

13.1 Weekly feature builder

```
import numpy as np, pandas as pd

COUNT_KINDS = {
    "commit": "n_commits",      "pr_opened": "n_prs_opened",
    "pr_merged": "n_prs_merged", "message": "n_messages",
    "doc_edited": "n_docs_edited", "decision": "n_decisions",
}

def build_weekly_features(events, tenant_id):
    ev = events.copy()
    ev["ts"] = pd.to_datetime(ev["ts"], utc=True)
    ev["week"] = ev["ts"].dt.to_period("W").dt.start_time
    hour = ev["ts"].dt.hour
    ev["after_hours"] = (hour < 9) | (hour >= 18)

    rows = []
    for week, g in ev.groupby("week"):
        n = len(g)
        row = {"week": week, "observed": 1}

        # Family 1 – counts
        for kind, col in COUNT_KINDS.items():
            row[col] = int((g["kind"] == kind).sum())

        # Family 2 – ratios
        row["after_hours_ratio"] = float(g["after_hours"].mean()) if n else 0.0
        row["merge_rate"] = _safe_div(row["n_prs_merged"], row["n_prs_opened"])

        # Family 3 – latencies (meta carries precomputed deltas)
        row["median_cycle_time"] = _median_meta(g, "cycle_time_s")

        # Family 4 – dispersion
        shares = g["actor"].value_counts(normalize=True).values
        row["actor_entropy"] = float(-(shares * np.log(shares + 1e-9)).sum())
        row["top_actor_share"] = float(shares.max()) if len(shares) else 0.0
        row["n_active_actors"] = int(g["actor"].nunique())
        rows.append(row)

    df = pd.DataFrame(rows).set_index("week").sort_index()

    # Rule 5 – complete the grid so gaps become EXPLICIT
    full = pd.date_range(df.index.min(), df.index.max(), freq="W-MON")
    df = df.reindex(full)
    df["observed"] = df["observed"].fillna(0).astype(int)

    gap = df["observed"].eq(0)
    df["weeks_since_last_event"] = gap.groupby((~gap).cumsum()).cumsum()
    df["tenant_id"] = tenant_id
    return df
```

13.2 Topic drift from existing embeddings

```
def weekly_text_features(week_embeddings, baseline_centroid, sentiments):
    out = {"sentiment_mean": 0.0, "sentiment_std": 0.0, "topic_drift": 0.0}
    if sentiments:
        out["sentiment_mean"] = float(np.mean(sentiments))
        out["sentiment_std"] = float(np.std(sentiments))
    if week_embeddings is not None and len(week_embeddings) \
        and baseline_centroid is not None:
        c = week_embeddings.mean(axis=0)
        c = c / (np.linalg.norm(c) + 1e-9)
        b = baseline_centroid / (np.linalg.norm(baseline_centroid) + 1e-9)
        out["topic_drift"] = float(1.0 - np.dot(c, b)) # cosine distance
    return out
```

13.3 Per-tenant normaliser (Rules 1, 2, 3, 6)

```
class TenantNormalizer:
    """Per tenant. Fit on the TRAINING WINDOW ONLY."""

    def __init__(self):
        self.stats = {}

    def fit(self, df, train_end):
        train = df.loc[:train_end].copy()
        for c in COUNT_COLS: # Rule 2
            if c in train: train[c] = np.log1p(train[c].fillna(0))
        for c in train.select_dtypes("number").columns:
            if c in PASSTHROUGH: continue
            self.stats[c] = (float(train[c].mean()), float(train[c].std()))
        return self

    def transform(self, df):
        out = df.copy()
        for c in COUNT_COLS:
            if c in out: out[c] = np.log1p(out[c].fillna(0))
        for c, (mu, sd) in self.stats.items(): # Rule 1 + 6
            if c in out: out[c] = (out[c] - mu) / (sd + 1e-6)

        w = pd.DatetimeIndex(out.index).isocalendar().week.values.astype(float)
        out["week_sin"] = np.sin(2 * np.pi * w / 52.0) # Rule 3
        out["week_cos"] = np.cos(2 * np.pi * w / 52.0)

        num = out.select_dtypes("number").columns
        out[num] = out[num].fillna(0.0)
        return out
```

13.4 Rolling-window self-labelling

```
def make_windows(arr, encoder_len, horizon):
    """The future values ARE the labels – every elapsed week turns a
    prediction into a labelled training example."""
    T = arr.shape[0]
    last_start = T - (encoder_len + horizon)
    for s in range(0, max(last_start + 1, 0)):
        x = arr[s : s + encoder_len]
        y = arr[s + encoder_len : s + encoder_len + horizon]
        yield x, y
```

14 · Where to Find Training Data

The forecasting engine trains on two fundamentally different data sources: your own internal signals (which you generate) and external country signals (which you source publicly). Here is exactly where each comes from.

14.1 Internal signals (Tier 1) — you generate these

This is the most important point: **your primary training data is your own product's pipeline output**. Every connector (Gmail, Slack, Notion, GitHub) already produces the time-series you need. You do not buy this — you accumulate it.

SIGNAL	DERIVED FROM	HOW TO COMPUTE
Comms volume	Gmail, Slack connectors	Count messages per team per week
After-hours ratio	message timestamps	% of activity outside working hours
Sentiment	message text	RoBERTa sentiment score, weekly mean
Task throughput	GitHub, Jira	Issues/PRs closed per week
Doc creation rate	Notion, Drive	New/edited docs per week
Decision frequency	Brain decisions log	Decisions detected per week

THE COLD-START REALITY

You start with zero internal history. Two paths: (1) bootstrap on public proxy datasets (below) to pretrain the sequence encoder, then fine-tune on real tenant data as it accumulates; (2) lean on static industry+geography priors for new tenants and widen confidence bands until their own history builds up. See Section 11.

14.2 External country signals (Tier 3) — public sources

All free, all official, all downloadable as time-series. Sync these on a schedule into your external signal store.

SIGNAL	SOURCE	ACCESS
Inflation / CPI	World Bank Open Data	data.worldbank.org · free API
Interest rates	Central bank publications / BIS	bis.org · free download
GDP growth	World Bank · IMF	data.worldbank.org · imf.org/data
FX rates / volatility	ECB, central banks, exchangerate.host	free APIs
Employment / labour	ILO, national statistics offices	ilostat.ilo.org · free
Political stability	World Bank Governance Indicators	info.worldbank.org/governance/wgi
Holidays / calendars	Nager.Date, national calendars	date.nager.at · free API

14.3 Public proxy datasets for pretraining

To pretrain the sequence encoder before you have tenant data, use public multivariate time-series datasets. These teach the model general temporal dynamics.

DATASET	USE	WHERE
M4 / M5 Competition	General forecasting pretraining	github.com/Mcompetitions
UCI ML Repository (time-series)	Multivariate sequence patterns	archive.ics.uci.edu
GitHub Archive	Real dev-activity time-series (proxy for tech-company signals)	gharchive.org
Enron email corpus	Org communication patterns (proxy for comms signals)	cs.cmu.edu/~enron
Kaggle HR / attrition sets	Attrition/burnout label proxies	kaggle.com/datasets
FRED economic data	Macro time-series for geo pretraining	fred.stlouisfed.org

14.4 Labels — the hard part

Forecasting is supervised: you need the future outcome to train against. Your labels come from your own historical data via a **rolling-window** setup: train on weeks 1–52, predict week 53–65, and the actual values of those weeks (once they arrive) are the labels. This means the system improves automatically as time passes — every week that elapses turns a prediction into a labelled training example.

THE DATA FLYWHEEL

Internal signals are self-labelling over time. Each forecast you make becomes a training example once the real outcome arrives. Combined with cross-tenant learning, this means the system gets more accurate the longer it runs and the more tenants you add — a compounding data advantage a single-company tool can never replicate.

15 · Tenant Isolation & Cold-Start

Two operational concerns the architecture must handle explicitly: keeping tenants cryptographically separate, and producing useful forecasts for brand-new tenants with no history.

15.1 Tenant isolation

Every store in the architecture is namespaced per tenant. The single global model is shared (that is the source of cross-tenant learning), but no raw data, sequence trajectory, or forecast ever crosses a tenant boundary.

LAYER	ISOLATION MECHANISM
Sequence store (LMDB)	Per-tenant encrypted database, separate keys
Feature store	tenant_id namespace prefix on every record
Forecast store	Namespace-scoped queries, RBAC enforced
Model	Shared weights only — learns patterns, never memorises raw tenant data
Serving API	User's tenant_id must match requested namespace

SHARED MODEL, ISOLATED DATA — HOW BOTH HOLD

The model learns generalisable patterns ("rising after-hours activity precedes attrition") across the fleet, expressed in its weights. It never stores a tenant's raw sequence. So two competing finance companies benefit from the same learned dynamics while their actual data remains cryptographically separate. This is the balance that makes multi-tenant forecasting both powerful and safe.

15.2 Cold-start — forecasting for a new tenant

A brand-new tenant has almost no X_t history. The architecture handles this gracefully because identity comes from static conditioning, not just history.

- Lean on static priors.** With little sequence data, the fusion engine weights the industry and geography signals more heavily — the model forecasts "what a typical finance company in Kenya looks like" until real data refines it.
- Widen the bands.** Low data means high uncertainty, and the quantile output should reflect that honestly — the Bear-Bull spread widens automatically when the model is uncertain, because the quantile loss learns wider spreads where errors are larger.
- Fleet-based warm start.** Because the model is global, a new tenant inherits everything learned from similar tenants (same industry, same region) from day one.
- Narrow over time.** As the tenant's own history accumulates via the rolling window, the encoder's sequence signal grows stronger and the bands tighten.

THE COLD-START ADVANTAGE

A single-company forecasting tool is useless on day one — it has no history to learn from. CoBrain forecasts usefully from the first week because it borrows the fleet's learned patterns, conditioned on the new tenant's industry and geography. This is a direct product advantage you can state on the website: "useful forecasts from day one."

15.3 Summary of the corrected design

The system takes your original air-gapped design and makes it executable and correct: the sequence contract is explicit, the industry mask gates the input, geography spans static and time-varying channels, tenants are isolated, and interpretability is a first-class output. The result is a single conditioned model that forecasts across industries and countries, produces honest quantile bands, explains itself, and improves as it runs.

16 · Folder Structure & Build Order

The forecasting engine lives in `m_learning/`. Every file listed below must exist. The build order below is not a suggestion — the data layer must be tested before any model code is written, because a leakage bug in normalisation produces a flawless backtest and a useless product.

16.1 Folder structure

```
m_learning/
├── README.md
├── requirements.txt
├── config.py                # all hyperparameters, single source
├── registry/
│   ├── tiers.py            # tier definitions, forecast targets
│   └── covariates.py        # static covariate schema + vocabularies
├── data/
│   ├── event_loader.py     # pull raw events from Postgres per tenant
│   ├── feature_builder.py  # events → weekly grid, 5 families
│   ├── text_features.py    # sentiment + topic drift
│   ├── external_signals.py # country signal ingest (Tier 3)
│   ├── normalizer.py       # per-tenant, train-window-only fit
│   └── dataset.py          # torch Dataset → model tensors
├── models/
│   ├── layers.py           # GLU, GRN, VariableSelectionNetwork
│   ├── static_encoder.py   # the four context vectors (Fix 7)
│   ├── industry_mask.py    # Tier 2
│   ├── geo_projector.py    # Tier 3 static half
│   ├── encoder.py          # Universal encoder: LSTM + attention
│   ├── fusion.py           # fusion + interpretability head
│   ├── quantile_head.py    # P10 / P50 / P90
│   └── tft.py              # full assembly
├── training/
│   ├── losses.py           # quantile (pinball) loss
│   ├── rolling_window.py   # self-labelling window generator
│   ├── trainer.py          # train loop, temporal validation
│   └── train.py            # CLI entrypoint
├── inference/
│   ├── predictor.py        # load checkpoint, run forecast
│   └── serve.py            # namespace-scoped forecast API
├── storage/
│   ├── feature_store.py    # namespaced feature read/write
│   ├── forecast_store.py   # namespaced forecast persistence
│   └── model_registry.py   # checkpoint versioning
├── scripts/
│   ├── build_features.py
│   ├── sync_external.py
│   └── backfill.py
└── tests/
    ├── test_features.py
    ├── test_normalization.py
    ├── test_model_shapes.py # assert EVERY tensor boundary
    └── test_tenant_isolation.py
```

16.2 Build order

#	STEP	GATE
1	config.py, requirements.txt, README.md	—
2	registry/covariates.py, registry/tiers.py	schemas before code
3	storage/feature_store.py	namespaced accessor
4	data/event_loader.py	—
5	data/feature_builder.py	—
6	data/text_features.py	—
7	data/normalizer.py	—
8	tests/test_features.py + test_normalization.py	MUST PASS before step 9
9	models/layers.py (GLU, GRN, VSN)	—
10	models/static_encoder.py	Fix 7
11	models/industry_mask.py, geo_projector.py	—
12	models/encoder.py	Fixes 1 and 2
13	models/fusion.py	Fix 5
14	models/quantile_head.py	Fix 6
15	models/tft.py	assembly
16	tests/test_model_shapes.py	assert every boundary
17	training/ — losses, rolling_window, trainer, train	temporal split only
18	data/external_signals.py + scripts/sync_external.py	Tier 3 live
19	inference/predictor.py, storage/forecast_store.py	—
20	inference/serve.py + test_tenant_isolation.py	cross-tenant read must raise
21	scripts/build_features.py, backfill.py	—

16.3 Non-negotiable rules

- One global model conditioned by static covariates. Never one model per tenant.
- z_t is a per-timestep sequence [B,T,d]. Never pool or flatten before fusion.
- The industry mask gates the encoder input, never its output.

- Geography has two channels: static `e_geo` and time-varying `g_t`.
- The static encoder initialises the LSTM hidden state.
- The fusion engine emits interpretability weights as a second output.
- The output head uses the last timestep. Never Flatten.
- Weekly buckets, 52-week lookback, 13-week horizon.
- Missing weeks are an explicit mask plus `weeks_since_last_event`. Never silent zero-fill, never carry-forward.
- Normalise per tenant: log1p on counts, z-score on that tenant's own history, statistics fitted on the training window only.
- Calendar time encoded cyclically as sin/cos pairs. Never raw integers.
- Categoricals go to embeddings. Never raw integer IDs into numeric channels.
- `author_id`, `repo_id`, `channel_id` are not static covariates. Use dispersion features.
- Text becomes a handful of scalars. Never 384 embedding dimensions as 384 variables.
- Validation split is temporal. Never random.
- Quantile output always: P10 / P50 / P90. Never a point estimate.
- Every store access requires a `tenant_id`. No exceptions.

END OF DESIGN SPECIFICATION

CoBrain Forecasting Engine · Version 2.0